System Architecture

1. System Context

```mermaid graph TD User[Beneficiary/Staff] —>|HTTPS| WebApp[Next.js Web App] WebApp -- >|REST| API[NestJS API] API —>|TCP| MySQL[(Central MySQL)] Node[Distribution Node] —>|HTTPS| API Node —>|Local| SQLite[(Edge SQLite)] API —>|Async| Worker[Sync/Reconciliation Worker] Worker —>|TCP| MySQL 2. Container & Component Architecture Web App: React/Next.js. Serves UI, handles public registration. API Server: NestJS. Handles REST, auth, state machine, online quotas. Sync Worker: NestJS microservice. Processes outbox, reconciles offline batches. Central MySQL: Source of truth for persons, lots, quotas, reconciled txns. Edge Node: Standalone binary/service. Runs local SQLite, signs tokens, manages local outbox. 3. Deployment Architecture Dev: Docker Compose (MySQL, API, Web, Worker). Prod: Kubernetes or Docker Swarm. API scaled horizontally. MySQL Primary/Replica. Edge nodes deployed as standalone services on local hardware. 4. Consistency & Partitioning (Quota Strategy) Problem: Disconnected nodes cannot share a real-time global counter. Solution: Quota Leases. Central server allocates a specific quota amount (e.g., 50kg) to Node A for Card X. Node B gets a separate lease. Sum of leases <= Total Card Quota. Nodes enforce against their lease. Prevents double-spending without real-time sync. 5. Architecture Decision Records (ADRs) ADR-01: Central MySQL + Edge SQLite. Decision: Avoids complex multi-master DB replication. Edge SQLite is durable but ephemeral to the central ledger. ADR-02: REST for Sync. Decision: Simpler than Kafka for this scale. Outbox pattern ensures durability. ADR-03: Ed25519 for Tokens. Decision: Fast, secure, small signatures. No need for complex PKI like X.509 for internal node signing.
```